Semana 10: Patrones Flyweight y Proxy — Optimización y Control de Acceso

Curso: Patrones de Diseño de Software (PIS1003)
Corte: 2 — Semana 10
Modalidad: Presencial — Aprendizaje Basado en Proyectos (ABPr)
Docente: Eliecer Montero Ojeda
Proyecto Integrador: Sistema de Gestión Académica con Arquitectura Basada en Patrones

---

1. CONEXIÓN CON LA SEMANA 9

1.1. Recordatorio: ¿dónde estamos?

En la Semana 9 profundizamos en **Decorator** (extensiones dinámicas aplicadas a reportes y validaciones) e introdujimos **Facade** (simplificación del acceso a subsistemas). Hoy cerramos el bloque de patrones estructurales con dos patrones que resuelven problemas **no funcionales**:

- **Flyweight:** optimización del uso de memoria mediante el compartir objetos.
- **Proxy:** control de acceso a objetos mediante un intermediario.

Semana	Patrones	Pregunta que responden
7	Adapter, Bridge	¿Cómo traduzco interfaces? ¿Cómo desacoplo abstracción de implementación?
8	Composite, Decorator	¿Cómo trato hojas y ramas uniformemente? ¿Cómo añado responsabilidades dinámicamente?
9	Decorator (prof.), Facade	¿Cómo aplico decoradores a reportes? ¿Cómo simplifico el acceso a subsistemas?
10	**Flyweight, Proxy**	**¿Cómo ahorro memoria con objetos compartidos? ¿Cómo controlo el acceso a objetos costosos o sensibles?**

1.2. Los dos problemas que resolveremos hoy

Problema A — Explosión de memoria en objetos repetidos: En el Sistema de Gestión Académica, cada estudiante tiene asignado un **horario semanal** con materias, docentes, aulas y bloques horarios. Si hay 5.000 estudiantes y cada uno guarda su propio objeto de cada materia (con nombre, código, créditos, docente, etc.), el sistema puede consumir cientos de MB innecesariamente porque **miles de estudiantes comparten las mismas materias**.

Problema B — Acceso controlado a operaciones costosas o sensibles: Generar un **certificado oficial** con firma digital requiere autenticación, validación de pagos, criptografía y auditoría. No cualquier usuario puede invocarlo, ni conviene ejecutarlo si no es necesario. Además, algunos reportes pesados tardan 3-5 segundos en generarse y sería bueno **cachearlos**.

Flyweight resuelve el Problema A. Proxy resuelve el Problema B.

BLOQUE 2 — PATRÓN FLYWEIGHT

2.1. Propósito

Intención GoF: *"Use sharing to support large numbers of fine-grained objects efficiently."*

Idea central: Cuando tenemos **miles o millones de objetos** con mucha información **intrínseca** (compartida) y poca **extrínseca** (única por instancia), podemos:

1. Extraer la parte compartida a un objeto **Flyweight** (inmutable, cacheado).
2. Guardar la parte variable fuera del flyweight (en el cliente o en un objeto contenedor).
3. Compartir instancias del flyweight entre múltiples clientes.

2.2. Motivación: el horario académico

En el Sistema de Gestión Académica:

- **500 materias** existen en el catálogo institucional.
- **5.000 estudiantes** están matriculados.
- Cada estudiante cursa **6 materias** en promedio.

Enfoque ingenuo: Cada estudiante guarda su propio objeto Materia con todos los datos (nombre, código, créditos, docente, descripción, bibliografía, objetivos). Total de objetos: 5.000 × 6 = **30.000 objetos** Materia con información duplicada.

Enfoque Flyweight: Solo existen **500 objetos** Materia compartidos. Cada estudiante guarda referencias a ellos. Total de objetos: **500 materias + 5.000 horarios** que contienen solo referencias + datos variables (día, hora, aula).

Ahorro estimado: ~98% en memoria para las materias.

2.3. Estructura aplicada al proyecto



2.4. Implementación en el proyecto integrador

Flyweight — Materia (inmutable):

```
package co.uts.academico.horarios;

public final class Materia {
    // Todos los campos son intrínsecos (compartidos) e inmutables
    private final String codigo;
    private final String nombre;
    private final int creditos;
    private final String docente;
    private final String descripcion;

    public Materia(String codigo, String nombre, int creditos,
                   String docente, String descripcion) {
        this.codigo = codigo;
        this.nombre = nombre;
        this.creditos = creditos;
        this.docente = docente;
        this.descripcion = descripcion;
    }

    public String getCodigo() { return codigo; }
    public String getNombre() { return nombre; }
    public int getCreditos() { return creditos; }
    public String getDocente() { return docente; }

    // Estado extrínseco se pasa como parámetro al método
    public void mostrar(String dia, String hora, String aula) {
        System.out.printf("[%s %s - Aula %s] %s (%s) - %d créditos%n",
            dia, hora, aula, nombre, codigo, creditos);
    }
}
```

Flyweight Factory:

```
package co.uts.academico.horarios;

import java.util.HashMap;
import java.util.Map;

public class MateriaFactory {
    private static final Map<String, Materia> cache = new HashMap<>();

    public static Materia getMateria(String codigo) {
        // Si ya existe en el cache, se retorna la misma instancia
        return cache.computeIfAbsent(codigo, c -> {
            // Simulación: en producción vendría de BD
            return new Materia(c, "Materia " + c, 3, "Docente " + c, "Descripción " + c);
        });
    }

    public static int getTotalInstancias() {
        return cache.size();
    }
}
```

Cliente — BloqueHorario (estado extrínseco):

```
package co.uts.academico.horarios;

public class BloqueHorario {
    // Referencia al flyweight (compartida)
    private final Materia materia;
    // Estado extrínseco (único por bloque)
    private final String dia;
    private final String hora;
    private final String aula;

    public BloqueHorario(String codigoMateria, String dia, String hora, String aula) {
        this.materia = MateriaFactory.getMateria(codigoMateria);
        this.dia = dia;
        this.hora = hora;
        this.aula = aula;
    }

    public void mostrar() {
        materia.mostrar(dia, hora, aula);
    }
}
```

Cliente — Main:

```
package co.uts.academico.horarios;

import java.util.ArrayList;
import java.util.List;

public class GeneradorHorarios {
    public static void main(String[] args) {
        List<BloqueHorario> horarios = new ArrayList<>();

        // 5.000 estudiantes, cada uno con 6 materias = 30.000 bloques
        String[] dias = {"Lunes", "Martes", "Miércoles", "Jueves", "Viernes"};
        String[] horas = {"07:00", "09:00", "11:00", "14:00", "16:00", "18:00"};

        for (int i = 0; i < 5_000; i++) {
            for (int j = 0; j < 6; j++) {
                String codigoMateria = "MAT-" + (j + 1);
                horarios.add(new BloqueHorario(
                    codigoMateria,
                    dias[j % 5],
                    horas[j % 6],
                    "Aula-" + (j + 1)
                ));
            }
        }

        System.out.println("Total de bloques horarios: " + horarios.size());
        System.out.println("Total de materias (flyweights): " + MateriaFactory.getTotalInstancias
());
        System.out.println("Ahorro estimado: " +
            (100 - (MateriaFactory.getTotalInstancias() * 100 / horarios.size())) + "%");

        // Mostrar solo los primeros 5 para no saturar
        horarios.stream().limit(5).forEach(BloqueHorario::mostrar);
    }
}
```

2.5. Consecuencias de Flyweight

Ventajas:

Ventaja	Explicación
Reducción drástica de memoria	De 30.000 objetos a 500 (98% de ahorro)
Mejora en el rendimiento de GC	Menos objetos → menos presión al recolector de basura
Compartición transparente	El cliente no sabe que comparte instancias
Aplicable a objetos inmutables	Ideal para catálogos, íconos, configuraciones

Desventajas:

Desventaja	Explicación
Complejidad adicional	Hay que separar estado intrínseco y extrínseco
Riesgo de bugs por mutabilidad	El flyweight debe ser **inmutable**; si se modifica, todos los clientes lo ven
Trade-off tiempo/memoria	Buscar en el cache tiene costo; en objetos muy pequeños puede no valer la pena
Debugging más difícil	Los objetos compartidos aparecen en múltiples stack traces

2.6. Cuándo usar Flyweight

Cuándo SÍ	Cuándo NO
Hay miles/millones de objetos similares	Hay pocos objetos
Gran parte del estado es compartible	Todo el estado es único
Los objetos son inmutables	Los objetos cambian con frecuencia
El ahorro de memoria justifica el costo	El ahorro es marginal

2.7. Ejemplos reales

- **Editores de texto:** cada carácter referencia un Character flyweight (fuente, tamaño) y guarda solo su posición.
- **Videojuegos:** árboles, enemigos y proyectiles comparten sprites y propiedades.
- **Navegadores:** imágenes y estilos CSS se comparten entre múltiples nodos del DOM.
- **Java** Integer.valueOf()**:** cachea valores entre -128 y 127.

BLOQUE 3 — PATRÓN PROXY

3.1. Propósito

Intención GoF: *"Provide a surrogate or placeholder for another object to control access to it."*

Idea central: Un Proxy es un objeto que **se hace pasar por otro** y controla el acceso a él. Se usa cuando:

- El objeto real es **costoso de crear** (lazy initialization).
- El objeto real está en **otra máquina** (proxy remoto).
- El acceso requiere **autenticación/autorización** (protection proxy).
- Necesitamos **cachear** resultados (caching proxy).
- Necesitamos **registrar/loguear** accesos (logging proxy).

3.2. Motivación: los certificados académicos

En el Sistema de Gestión Académica, la generación de un **certificado oficial** implica:

1. Validar identidad del solicitante.
2. Verificar que esté al día en pagos.
3. Obtener notas y promedio.
4. Generar PDF con membrete.
5. Aplicar firma digital.
6. Registrar en auditoría.

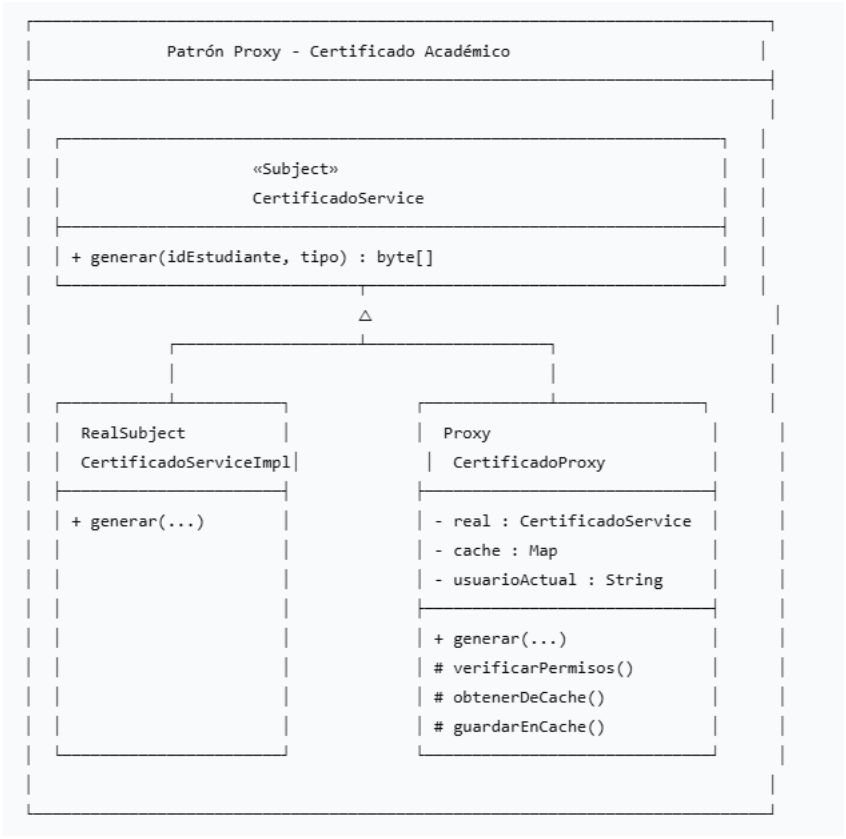
Esto tarda 3-5 segundos y es **sensible** (no cualquiera puede pedirlo). Además, si el mismo estudiante pide el mismo certificado dos veces en poco tiempo, conviene **cachear** el resultado.

3.3. Tipos de Proxy que aplicaremos

En el proyecto aplicaremos **tres tipos de proxy**:

1. **Protection Proxy:** verifica que el usuario tenga permiso.
2. **Virtual Proxy:** retrasa la creación del objeto costoso (lazy).
3. **Caching Proxy:** guarda el resultado para no recalcular.

3.4. Estructura aplicada al proyecto



3.5. Implementación en el proyecto integrador

Subject — Interfaz común:

```java
package co.uts.academico.certificados;

public interface CertificadoService {
    byte[] generar(int idEstudiante, String tipo);
}
```

RealSubject — Implementación costosa:

```java
package co.uts.academico.certificados;

public class CertificadoServiceImpl implements CertificadoService {
    @Override
    public byte[] generar(int idEstudiante, String tipo) {
        System.out.println("⏳ Generando certificado (operación costosa)...");
        simularTrabajo(3000); // 3 segundos
        System.out.println("✓ Certificado generado");
        return ("CERTIFICADO-" + tipo + "-" + idEstudiante).getBytes();
    }

    private void simularTrabajo(long ms) {
        try { Thread.sleep(ms); } catch (InterruptedException e) { }
    }
}
```

Proxy — Protection + Caching:

```java
package co.uts.academico.certificados;

import java.util.HashMap;
import java.util.Map;

public class CertificadoProxy implements CertificadoService {
    private final CertificadoService real;
    private final Map<String, byte[]> cache = new HashMap<>();
    private final String usuarioActual;
    private final String rolActual;

    public CertificadoProxy(String usuarioActual, String rolActual) {
        this.usuarioActual = usuarioActual;
        this.rolActual = rolActual;
        this.real = new CertificadoServiceImpl(); // Lazy: se puede diferir
    }
```

```
    @Override
    public byte[] generar(int idEstudiante, String tipo) {
        // 1. PROTECTION: verificar permisos
        if (!tienePermiso(idEstudiante)) {
            throw new SecurityException(
                "Usuario " + usuarioActual + " no autorizado para generar certificados de " + idEstu
diante);
        }

        // 2. CACHING: verificar si ya existe
        String claveCache = idEstudiante + ":" + tipo;
        if (cache.containsKey(claveCache)) {
            System.out.println(" ⚡ Certificado recuperado del cache");
            return cache.get(claveCache);
        }

        // 3. Delegar al objeto real (virtual proxy: solo si es necesario)
        byte[] resultado = real.generar(idEstudiante, tipo);

        // 4. Guardar en cache
        cache.put(claveCache, resultado);
        return resultado;
    }
```

```
   private boolean tienePermiso(int idEstudiante) {
       // Estudiante solo puede generar sus propios certificados
       if (rolActual.equals("ESTUDIANTE")) {
           return usuarioActual.equals("est-" + idEstudiante);
       }
       // Docentes y admin pueden generar cualquiera
       return rolActual.equals("DOCENTE") || rolActual.equals("ADMIN");
   }
}
```

Cliente:

```java
package co.uts.academico.certificados;

public class PortalCertificados {
    public static void main(String[] args) {
        // Caso 1: Estudiante genera su propio certificado
        CertificadoService proxy = new CertificadoProxy("est-123", "ESTUDIANTE");
        System.out.println("=== Primera solicitud (estudiante 123) ===");
        byte[] cert = proxy.generar(123, "ESTUDIOS");
        System.out.println("Tamaño: " + cert.length + " bytes");

        // Caso 2: Segunda solicitud idéntica → debe usar cache
        System.out.println("\n=== Segunda solicitud idéntica (debe usar cache) ===");
        proxy.generar(123, "ESTUDIOS");

        // Caso 3: Estudiante intenta generar certificado de otro
        System.out.println("\n=== Intento de acceso no autorizado ===");
        try {
            proxy.generar(456, "ESTUDIOS");
        } catch (SecurityException e) {
            System.out.println("✗ Bloqueado: " + e.getMessage());
        }
    }
}
```

3.6. Variantes de Proxy en el proyecto

Tipo de Proxy	Uso en el proyecto
Protection Proxy	CertificadoProxy verifica permisos por rol
Virtual Proxy	Reportes pesados que solo se generan cuando se solicitan
Caching Proxy	Cache de certificados y boletines
Remote Proxy	Integración con servicios externos (biblioteca, pagos)
Logging Proxy	Registro de auditoría de accesos sensibles

3.7. Consecuencias de Proxy

Ventajas:

Ventaja	Explicación
Control de acceso transparente	El cliente usa la misma interfaz; el proxy decide
Reducción de costos	Lazy loading: el objeto real se crea solo si se necesita
Mejora de rendimiento	Caching evita recomputar resultados costosos
Auditoría centralizada	Todos los accesos pasan por el proxy
Desacoplamiento	El cliente no conoce el objeto real

Desventajas:

Desventaja	Explicación
Latencia añadida	Cada llamada pasa por una capa extra
Complejidad	Más clases que mantener
Riesgo de staleness	El cache puede servir datos obsoletos
Debugging difícil	Los stack traces se alargan

3.8. Proxy vs. Decorator vs. Adapter

Aspecto	Proxy	Decorator	Adapter
Propósito	Controlar acceso	Añadir responsabilidades	Traducir interfaz
Interfaz	Igual al Subject	Igual al Component	Diferente al Adaptee
Crea el objeto real	Sí (o lo conoce)	No (lo recibe)	No (lo recibe)
Cambia comportamiento	Sí (controla)	Sí (extiende)	No (traduce)
Cliente sabe	No	No	Sí (o no, según diseño)

Regla mnemotécnica: *Proxy controla, Decorator enriquece, Adapter traduce*.

3.9. Ejemplos reales

- **Spring AOP:** @Transactional, @Cacheable, @PreAuthorize se implementan con proxies dinámicos.
- **Hibernate:** proxies de entidades para lazy loading.
- **Java RMI:** proxies remotos para llamar métodos en otra JVM.
- **Nginx/HAProxy:** proxy inverso a nivel de red.

BLOQUE 4 — COMPARATIVA FINAL DE LOS SIETE PATRONES ESTRUCTURALES

4.1. Tabla comparativa completa

Patrón	Intención	¿Envuelve?	¿Cambia interfaz?	¿Añade comportamiento?	Cuándo usarlo
Adapter	Traducir interfaces	1 objeto	Sí	No	Interfaces incompatibles
Bridge	Desacoplar abstracción/ implementación	1 objeto	No	No	Múltiples dimensiones ortogonales
Composite	Tratar hojas y ramas uniformemente	Muchos	No	No	Jerarquías parte-todo
Decorator	Añadir responsabilidades dinámicamente	1 objeto	No	Sí	Extensiones combinables
Facade	Simplificar acceso a subsistemas	Muchos	Sí (simplifica)	No	Subsistemas complejos

Patrón	Intención	¿Envuelve?	¿Cambia interfaz?	¿Añade comportamiento?	Cuándo usarlo
Flyweight	Compartir objetos para ahorrar memoria	0 (comparte)	No	No	Muchos objetos similares
Proxy	Controlar acceso a un objeto	1 objeto	No	Sí (controla)	Lazy, seguridad, cache

4.2. Regla mnemotécnica global

- **Adapter** = traductor (cambia interfaz)
- **Bridge** = arquitecto (separa jerarquías)
- **Composite** = árbol (trata hojas y ramas igual)
- **Decorator** = capas (añade sin cambiar)
- **Facade** = recepcionista (un punto de entrada)
- **Flyweight** = biblioteca compartida (ahorra memoria)
- **Proxy** = guardaespaldas (controla el acceso)

4.3. Combinación de los siete patrones en el proyecto integrador

Un diseño maduro del Sistema de Gestión Académica combina:

Módulo	Patrón	Aplicación
Estructura académica	**Composite**	Facultad → Programa → Curso → Estudiante
Reportes	**Decorator**	Encabezado, firma, marca de agua, notas al margen

Módulo	Patrón	Aplicación
Portal del estudiante	**Facade**	Un único punto de entrada a 5 subsistemas
Catálogo de materias	**Flyweight**	500 materias compartidas entre 5.000 estudiantes
Certificados y reportes pesados	**Proxy**	Control de acceso + cache + lazy loading
Integración con pasarelas de pago	**Adapter**	Unificar PayU, Mercado Pago, Efecty
Notificaciones multicanal	**Bridge**	Email, SMS, push con implementaciones intercambiables

---

BLOQUE 5 — CIERRE Y ACTIVIDADES

5.1. Preguntas para la reflexión

1. **¿Por qué Flyweight exige que el objeto compartido sea inmutable?** (Si se modifica, todos los clientes ven el cambio; se rompe la semántica de "objeto compartido").
2. **¿Qué diferencia hay entre un Proxy y un Decorator si ambos envuelven un objeto?** (Proxy **controla** el acceso; Decorator **añade** responsabilidades).
3. **¿Cuándo NO conviene usar Flyweight?** (Cuando el estado intrínseco es pequeño o los objetos se modifican con frecuencia).
4. **¿Cómo se relaciona Proxy con el principio Open/Closed?** (Se agregan proxies sin modificar el objeto real ni el cliente).
5. **¿Qué patrón usarías para cachear resultados de una API externa?** (Proxy + Caching).

6. **¿Cómo se combinan Flyweight y Singleton?** (El cache del Flyweight suele ser un Singleton).

---

6. REFERENCIAS BIBLIOGRÁFICAS

Bibliografía básica:

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
    - **Flyweight:** pp. 195–206
    - **Proxy:** pp. 207–217
- Freeman, E., & Robson, E. (2020). *Head First Design Patterns* (2nd ed.). O'Reilly. (Cap. 11: Proxy; Cap. 12: Compound Patterns)
- Martin, R. C. (2002). *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall. (Cap. 8–12: SOLID)

Webgrafía:

- Refactoring.Guru. (2023). *Flyweight*. https://refactoring.guru/design-patterns/flyweight
- Refactoring.Guru. (2023). *Proxy*. https://refactoring.guru/design-patterns/proxy
- SourceMaking. (2023). *Flyweight Pattern*. https://sourcemaking.com/design_patterns/flyweight
- SourceMaking. (2023). *Proxy Pattern*. https://sourcemaking.com/design_patterns/proxy
- Oracle. (2024). *Java SE Documentation: Integer Cache*. https://docs.oracle.com/javase/

Software y herramientas:

- Java 17 + IntelliJ IDEA/Eclipse
- PlantUML para diagramas de clases
- Git/GitHub para control de versiones
- JUnit 5 para pruebas
- VisualVM para analizar el uso de memoria

7.2. Plantilla de ADR

markdown

```
# ADR 0010: Uso de Flyweight y Proxy en el Sistema de Gestión Académica

## Estado
Aceptado

## Contexto
El sistema maneja 5.000 estudiantes x 6 materias = 30.000 objetos de materia,
y requiere control de acceso + cache para certificados costosos.

## Alternativas consideradas
1. Crear un objeto Materia por cada estudiante (problema: 30.000 objetos duplicados)
2. Usar un Singleton de catálogo global (problema: no distingue estados extrínsecos)
3. Caching manual disperso (problema: código duplicado, difícil de mantener)
4. Flyweight + Proxy (elegida)

## Decisión
Usar **Flyweight** para el catálogo de materias y **Proxy** (protección + cache) para certificados.

## Consecuencias
- Positivas: ahorro ~98% de memoria, control de acceso centralizado, mejora en rendimiento
- Negativas: complejidad adicional, riesgo de staleness en cache, debugging más difícil

## Referencias
- Gamma et al. (1994), Flyweight pp. 195–206, Proxy pp. 207–217.
```

7.3. Preguntas de evaluación formativa

1. ¿Por qué el estado extrínseco **no** se guarda en el Flyweight?
2. ¿Qué diferencia hay entre Flyweight y un cache tradicional?
3. ¿Cuántos tipos de Proxy conoces? Da un ejemplo de cada uno.
4. ¿Cómo se relaciona Proxy con el principio de responsabilidad única?
5. ¿Por qué Spring usa proxies para @Transactional?

8. RESUMEN DE LA SEMANA 10

Concepto	Clave
Flyweight	Compartir objetos inmutables para ahorrar memoria en sistemas con miles de instancias
Proxy	Intermediario que controla el acceso a un objeto (protección, lazy, cache, logging)
Cuándo Flyweight	Muchos objetos similares, estado mayormente compartible, objetos inmutables
Cuándo Proxy	Objetos costosos, sensibles, remotos o que se benefician de cache
Diferencia clave	Flyweight **comparte**; Proxy **controla**
Aplicación en proyecto	Catálogo de materias compartido + certificados con control de acceso y cache
Combinación	Flyweight + Proxy + Singleton suelen aparecer juntos en sistemas con caches

Concepto	Clave
Principios SOLID	Flyweight favorece responsabilidad única; Proxy favorece Open/Closed

---

9. CIERRE DEL BLOQUE ESTRUCTURAL

Con la Semana 10 **cerramos el bloque de patrones estructurales**. Los siete patrones vistos (Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy) forman el **vocabulario completo** para resolver problemas de composición, extensión, simplificación, optimización y control de acceso.